

1) What is Java?

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. It is widely used for building cross-platform applications, from mobile apps (Android) to web apps, enterprise systems, and more.

2) Explain how Java is platform-independent or WORA (Write Once, Run Anywhere) Architecture?

Java is platform-independent due to its "WORA" philosophy. This is achieved by compiling Java code into bytecode, which is then executed by the Java Virtual Machine (JVM) rather than directly by the underlying operating system. Since the JVM is available for various platforms, Java code can run on any system with a JVM, making it truly cross-platform.

3) What is bytecode?

Bytecode is an intermediate, platform-independent code generated by the Java compiler from Java source code. It's not machine code but is executed by the JVM. When you run a Java program, the JVM reads and interprets this bytecode.

4) Explain Java Compilation?

Java compilation refers to converting Java source code into bytecode using the Java compiler (javac). The source code, written in .java files, is compiled into .class files, which contain the bytecode. The JVM can then execute this bytecode.

5) Is JRE Platform Independent?

 Different JRE for Different OS so JVM AND JRE IS DEPENDENT
As the **Java Runtime Environment (JRE)** is platform-independent. It consists of the JVM and libraries required to run Java programs. The JRE makes Java applications executable on any platform with a compatible JVM.

6) Is JDK Platform Independent?

Yes, the **Java Development Kit (JDK)** is also [Platform dependent](#) platform-independent. It includes tools (compiler, debugger, etc.) for developing Java applications. The JDK can be installed on any system that supports the JRE.

7) What are Object-Oriented Concepts?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects." These objects have **attributes** (data) and **methods** (functions). The core concepts of OOP are:

1. **Encapsulation** – Wrapping data and methods in a single unit (class).
2. **Inheritance** – Deriving new classes from existing ones.
3. **Polymorphism** – The ability to take many forms, typically through method overloading or overriding.
4. **Abstraction** – Hiding implementation details and showing only essential features.

8) What are Object-Oriented Principles?

The main principles are:

1. **Encapsulation** – Protecting data and making it accessible only through methods.
2. **Inheritance** – A mechanism to define a new class based on an existing class.
3. **Polymorphism** – Allowing objects to be treated as instances of their parent class, but each object behaves differently.
4. **Abstraction** – Hiding unnecessary details and exposing only what is necessary.

9) Is Java 100% Object-Oriented?

No, Java is not 100% object-oriented. For instance, **primitive data types** like int, float, char, etc., are not objects. But the language promotes the use of objects for most operations.

10) What is a Variable and Types of Variables?

A variable is a storage location in memory, identified by a name, that holds a value.

Types of variables in Java:

- **Instance variables** – Declared inside a class but outside methods; they are unique to each object.
- **Class variables** (static) – Declared with the static keyword; shared among all instances of a class.
- **Local variables** – Declared within methods or constructors; they exist only within the scope of the method.

11) How do we initialize the non-static variables?

Non-static variables (instance variables) can be initialized in constructors, or directly when declared in the class.

```
class Example {  
    int x = 10; // direct initialization  
  
    Example() {  
        x = 20; // initialized in constructor  
    }  
}
```

12) Differentiate between local and global variables?

- **Local variables:** Declared inside methods, constructors, or blocks. They are not accessible outside their scope.

- **Global variables:** Also known as **instance variables**, declared inside a class but outside methods, making them accessible to all methods in that class.

13) Differentiate between static and non-static variables?

- **Static variables:** Declared with the static keyword. They are shared among all instances of the class.
- **Non-static variables:** Unique to each object of the class.

14) Differentiate between static and non-static methods?

- **Static methods:** Belong to the class, not instances. Can be called without creating an object.
- **Non-static methods:** Belong to an object of the class. They require an instance to be invoked.

15) Can we make local variables private?

No, **local variables** can't be declared private since they are already limited in scope to the method they are defined in.

16) Can we create a local variable as final?

Yes, you can declare **local variables** as final. This means they cannot be modified after initialization.

17) Can we create a global variable as final without initializing?

No, **final variables** must be initialized before they are used. If declared final, they must be assigned a value either during declaration or in the constructor.

18) Differentiate between print and println statements?

- **print():** Prints the message but does not move to the next line.
- **println():** Prints the message and moves the cursor to the next line.

19) Can we achieve Narrowing implicitly?

No, **narrowing** (casting from a larger type to a smaller type) requires explicit casting, as data may be lost.

20) Are true, false, and null keywords?

- **true** and **false** are **boolean literals** in Java, not keywords.
- **null** is a special literal but not a keyword.

21) Differentiate between constructor and method?

- **Constructor:** Special method to initialize an object. It has the same name as the class and no return type.
- **Method:** A block of code that performs an action. It can have a return type and can be called multiple times.

22) Differentiate between final Method and private method?

- **final method:** Cannot be overridden by subclasses.
- **private method:** Can only be accessed within the class where it is defined.

23) Explain Constructor with a real-time example?

A constructor is used to initialize an object. For example:

```
class Car {
    String model;

    Car(String model) { // Constructor
        this.model = model;
    }

    void display() {
        System.out.println("Car model: " + model);
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Tesla"); // Constructor called
```

```
        myCar.display();
    }
}
```

24) Explain Constructor Overloading with a real-time example?

Constructor overloading allows multiple constructors with different parameters.

```
class Car {
    String model;
    int year;

    Car(String model) {
        this.model = model;
    }

    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    void display() {
        System.out.println("Car model: " + model + ", Year: " + year);
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Tesla");
        Car myOldCar = new Car("Ford", 2015);
    }
}
```

```
    myCar.display();  
    myOldCar.display();  
}  
}
```

25) Which is the most used OOP's concept?

The most commonly used OOP concept is **Inheritance**. It allows code reuse, meaning you can extend existing classes and add or modify functionality without altering the original code. This promotes cleaner, more maintainable code.

26) Where do we find all inbuilt classes and interfaces of Java?

All inbuilt classes and interfaces are found in the **Java Standard Library** (also called the **Java API**). These are packaged into libraries (also known as JAR files) which come with the Java Development Kit (JDK). Some common packages include `java.util`, `java.io`, `java.lang`, etc.

27) Differentiate between Free Software and Open Source?

- **Free Software:** Refers to software that grants users the **freedom** to run, modify, and distribute it. Free software is not necessarily free in price, but it is free in terms of liberty (as defined by the Free Software Foundation).
- **Open Source:** Refers to software that has its **source code** made available for modification or enhancement. Open source software can be free or paid, but the defining feature is accessibility to the source code.

28) Can we convert bytecode into Java Source Code?

It is not possible to **directly** convert bytecode back to the original Java source code because information like variable names, comments, and other high-level constructs are lost during compilation. However, tools like **decompilers** can attempt to reverse-engineer bytecode into a readable

form of code, but the result won't be exactly the same as the original source code.

29) Is it possible to import two classes with the same name but from different packages?

Yes, it's possible to import two classes with the same name from different packages using **fully qualified names**:

```
import package1.ClassName;  
import package2.ClassName; // This would cause a conflict
```

// Resolve conflict using fully qualified names:

```
package1.ClassName object1 = new package1.ClassName();  
package2.ClassName object2 = new package2.ClassName();
```

30) Name some inbuilt packages present in Java?

Here are a few commonly used Java packages:

- java.lang – Basic classes like String, Object, Math, etc.
- java.util – Collections framework, utilities like ArrayList, HashMap, etc.
- java.io – Input/output classes like File, BufferedReader, etc.
- java.net – Networking classes like URL, Socket, etc.
- java.sql – JDBC classes for database connectivity.

31) What is Inheritance? Explain with a real-time example?

Inheritance allows one class to inherit fields and methods from another class. It promotes code reusability.

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```



```

}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited from Animal class
        dog.bark();
    }
}

```

32) Advantages of Inheritance/ Why inheritance is required?

- **Code Reusability:** Inheritance allows you to reuse existing code by inheriting properties and methods from a superclass.
- **Extensibility:** It makes it easier to extend and maintain systems by adding new functionality without modifying existing code.
- **Polymorphism:** You can use parent class references to refer to child class objects, enabling method overriding.

33) Is Multiple Inheritance Possible in Java?

yes possible

~~No~~, Java does not support **multiple inheritance** with classes to avoid complexity and ambiguity. However, you can achieve multiple inheritance through **interfaces**.

34) Why Multiple Inheritance is not possible in Java using classes?

Multiple inheritance can cause issues like the **Diamond Problem**, where ambiguity arises if a subclass inherits the same method from more than one parent class. Java avoids this by not allowing multiple inheritance through classes.

35) How to avoid inheritance in Java?

To avoid inheritance, you can declare a class as **final**. A final class cannot be subclassed.

```
final class FinalClass {  
    // code here  
}
```

36) Why Constructor Chaining? Advantages of Constructor Chaining?

Constructor chaining is when a constructor calls another constructor within the same class or in a superclass. It's used to avoid redundancy and reuse constructor logic.

Advantages:

- **Code Reusability:** You don't have to write initialization code repeatedly.
- **Cleaner Code:** Reduces complexity and improves code maintainability.

Example:

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}
```

```
class Dog extends Animal {  
    Dog() {  
        super(); // Calls the parent class constructor  
    }  
}
```

```
        System.out.println("Dog constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog(); // Calls both Dog and Animal constructors
    }
}
```

37) What is Method Overriding? Explain with a real-time example?

Method overriding occurs when a subclass provides its own implementation of a method that is already defined in its superclass. This allows polymorphic behavior.

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
```

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        Animal dog = new Dog(); // Polymorphism  
  
        animal.sound();  
        dog.sound(); // Calls Dog's sound() method  
    }  
}
```

38) Differentiate between Method Overloading and Method Overriding?

- **Method Overloading:** Happens within the same class. It involves defining multiple methods with the same name but different parameters (either in type, number, or both).
- **Method Overriding:** Happens between parent and child classes. It involves redefining a method from the parent class with the same signature in the child class.

39) Explain Method Overloading with real-time examples?

Method overloading allows a class to have more than one method with the same name, but different parameter types or numbers.

Example:

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

```

    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));    // Calls add(int, int)
        System.out.println(calc.add(2.5, 3.5)); // Calls add(double, double)
    }
}

```

40) Can we overload an abstract method?

Yes, you can **overload** an abstract method in Java. Just like any regular method, an abstract method can have multiple versions with different parameters in the same class or interface.

Example:

```

abstract class Animal {
    abstract void sound();

    // Overloaded abstract method
    abstract void sound(String noise);
}

```

```

class Dog extends Animal {
    @Override
    void sound() {

```

```

        System.out.println("Bark!");
    }

    @Override
    void sound(String noise) {
        System.out.println("Dog makes a " + noise + " sound.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();        // Bark!
        dog.sound("woof");  // Dog makes a woof sound.
    }
}

```

41) Is Overloading possible in two different classes?

Yes, overloading can happen in different classes. Method overloading refers to defining multiple methods with the same name but different parameters, and this can happen in any class, even across inheritance hierarchies.

Example:

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}

```

```
class AdvancedCalculator extends Calculator {  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        AdvancedCalculator calc = new AdvancedCalculator();  
        System.out.println(calc.add(2, 3));    // Inherited method  
        System.out.println(calc.add(2.5, 3.5)); // Overloaded method  
    }  
}
```

42) Differentiate between **this()** and **super()**

- **this()**: Refers to the current instance of the class. It is used to call the current class's constructor.
- **super()**: Refers to the superclass of the current class. It is used to call a constructor or method from the parent class.

Example:

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}
```

```
class Dog extends Animal {  
    Dog() {
```

```

        super(); // Calls Animal constructor
        System.out.println("Dog constructor");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
    }
}

```

43) Differentiate between this and super?

- **this**: Refers to the current object (instance of the class). It is used to access instance variables, methods, and constructors in the current class.
- **super**: Refers to the superclass (parent class) object. It is used to access superclass methods or variables, or to call the superclass constructor.

44) Why main method is static?

The **main()** method is static so that it can be called without creating an instance of the class. The Java runtime system calls the **main()** method to start the execution of a program. If it wasn't static, we'd need to create an object of the class to invoke **main()**, which wouldn't be practical.

45) Why main method contains String[] args why not int[] or Object[]?

The **String[] args** is used to pass command-line arguments to the Java application when it starts. These arguments are usually passed as strings, which is why it's a **String[]**. You could technically use any array type, but **String[]** is the standard.

46) What are the access modifiers for the class?

There are four main access modifiers in Java:

- **public:** The class is accessible from anywhere.
- **protected:** The class is accessible within the same package and to subclasses.
- **default** (no modifier): The class is accessible within the same package.
- **private:** Classes cannot be private. This modifier is used for methods or variables, not the class itself.

47) `System.out.println(null)`? Output?

The output will be null. Java can print null as a string when it encounters it.

Example:

```
System.out.println(null); // Outputs: null
```

48) Explain method overriding rules?

Rules for method overriding:

1. The method in the subclass must have the same signature as the method in the superclass.
2. The overridden method in the subclass should have the same return type or a subtype (covariant return).
3. You cannot reduce the visibility of the overridden method (e.g., you can't override a public method with a private one).
4. The method in the subclass can throw fewer or the same exceptions as the superclass method (or none).

49) Can we overload main method?

Yes, you can overload the `main()` method like any other method in Java, though it's not commonly done. This won't change the way the JVM starts your program, as it always calls the standard `main()` method.

Example:

```
class Main {
    public static void main(String[] args) {
```

```

        System.out.println("Main with String[] args");
    }

    public static void main(int a) {
        System.out.println("Main with int parameter: " + a);
    }
}

public class Test {
    public static void main(String[] args) {
        Main.main(10); // Calls overloaded main method with int parameter
    }
}

```

50) Can we override main method?

Yes, you can technically **override** the main() method in a subclass, but it is not recommended, as the main() method is the entry point for your program and is **static**. Static methods cannot be overridden in the usual sense. However, the subclass can **hide** the main method by defining its own static main().

51) Can we make public static void main to static public void main, will it work?

Yes, the order of the modifiers in the method declaration doesn't matter. Both public static void main and static public void main are valid and will work the same way.

52) Can we explicitly invoke main() method?

Yes, you can explicitly invoke the main() method like any other static method.

Example:

```

public class Main {

```

```
public static void main(String[] args) {  
    System.out.println("Main method called explicitly");  
}  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Main.main(args); // Explicitly calling the main method of Main class  
    }  
}
```

53) What are the access modifiers used for constructors?

The access modifiers used for constructors are the same as for methods:

- **public**: The constructor is accessible from anywhere.
- **protected**: The constructor is accessible within the same package and to subclasses.
- **default**: The constructor is accessible within the same package.
- **private**: The constructor is accessible only within the class, useful for Singleton patterns.

54) Can we define a constructor inside an interface?

No, you cannot define a constructor inside an **interface**. Interfaces cannot have constructors because they cannot be instantiated directly. The purpose of an interface is to define abstract behaviors, not to instantiate objects.

55) Why can't we create an object for an interface?

Interfaces cannot be instantiated because they are abstract by nature, meaning they only provide method declarations without providing implementation. You must implement the interface in

a concrete class to create an object.

56) Why can't we create an object for an abstract class?

An abstract class cannot be instantiated because it might have abstract methods that are not implemented. You need to create an object of a subclass that provides implementations for the abstract methods.

57) How to initialize the non-static variable present in the abstract class?

Non-static variables in an abstract class can be initialized using a constructor or directly when they are declared.

Example:

```
abstract class Animal {
```

```
    String name;
```

```
    Animal(String name) {
```

```
        this.name = name;
```

```
    }
```

```
}
```

```
class Dog extends Animal {
```

```
    Dog(String name) {
```

```
        super(name);
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Dog name: " + name);
```

```
    }
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog dog = new Dog("Buddy");  
        dog.display();  
    }  
}
```

58) Types of Interfaces?

- **Marker Interface:** An interface with no methods (e.g., Serializable).
- **Functional Interface:** An interface with just one abstract method (e.g., Runnable).
- **Regular Interface:** An interface that can have multiple abstract methods.

59) Name some Marker Interfaces?

- Serializable
- Cloneable
- Remote

60) Name some Functional Interfaces?

- Runnable
- Callable
- Comparator
- Predicate
- Function

Questions related to Exception

1) What is an exception?

An **exception** is an **event** that disrupts the normal flow of the program's execution. It indicates that something unexpected has occurred, and it can either be handled or propagated to the calling method. Exceptions can occur due to various reasons, such as invalid input, file not found, or database connectivity issues.

2) Can an exception occur at Compile Time?

Yes, **compile-time exceptions** are called **checked exceptions**. These are exceptions that occur during the compilation process. They must be explicitly handled using try-catch blocks or declared in the method signature using throws. Example: IOException, SQLException.

3) Differentiate between Exception and Error?

- **Exception:** These are conditions that a program can handle using try-catch blocks or propagate them. Examples: NullPointerException, FileNotFoundException.
 - **Error:** These are severe conditions that generally cannot be handled by the program and indicate problems with the JVM itself. Examples: OutOfMemoryError, StackOverflowError.
-

4) What are the types of Exception?

Exceptions in Java are categorized as:

1. **Checked Exceptions:** These exceptions are checked at compile-time. They must be either caught or declared to be thrown. Example: `IOException`, `ClassNotFoundException`.
 2. **Unchecked Exceptions:** These exceptions are not checked at compile time. They are runtime exceptions. Example: `NullPointerException`, `ArithmeticException`.
-

5) Name few exceptions that you experience in your project?

Some common exceptions I might have encountered in a typical Java project:

- `NullPointerException`: Trying to reference a null object.
 - `FileNotFoundException`: Trying to access a file that doesn't exist.
 - `ArrayIndexOutOfBoundsException`: Accessing an array element with an invalid index.
 - `SQLException`: When there is an issue with database connectivity.
 - `ClassNotFoundException`: When a class is not found at runtime.
-

6) Write Exception Hierarchy?

Here's the general exception hierarchy in Java:

- **Throwable**
 - **Error**
 - `OutOfMemoryError`, `StackOverflowError`, etc.
 - **Exception**
 - **Checked Exceptions:** `IOException`, `SQLException`, etc.
 - **Unchecked Exceptions:** `RuntimeException`
 - `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, etc.
-

7) How do you handle an Exception?

You can handle exceptions using try-catch blocks:

```
try {  
    // Code that may throw an exception  
} catch (ExceptionType e) {  
    // Handling the exception  
} finally {  
    // Optional block, always executed  
}
```

8) Explain Null Pointer Exception?

A **NullPointerException** occurs when you try to use a reference variable that points to **null**, meaning it doesn't refer to any object. This is one of the most common runtime exceptions in Java.

Example:

```
String str = null;  
System.out.println(str.length()); // Throws NullPointerException
```

9) How do you handle Null Pointer Exception?

You can handle a **NullPointerException** by checking if the object is null before using it:

```
String str = null;  
if (str != null) {  
    System.out.println(str.length());  
} else {  
    System.out.println("Object is null");  
}
```


Alternatively, you can handle it with a try-catch block:

```
try {  
    System.out.println(str.length());  
} catch (NullPointerException e) {  
    System.out.println("NullPointerException caught");  
}
```

10) Differentiate between checked and unchecked Exception?

- **Checked Exceptions:** These are exceptions that are checked at **compile-time**. You must either catch or declare them in the method signature using throws. Example: IOException, SQLException.
 - **Unchecked Exceptions:** These are exceptions that occur at **runtime** and are subclasses of RuntimeException. They do not need to be caught or declared. Example: NullPointerException, ArrayIndexOutOfBoundsException.
-

11) Differentiate between throw and throws keyword?

- **throw:** Used to **explicitly throw** an exception within a method. You use it to trigger an exception.

Example:

```
if (age < 18) {  
    throw new IllegalArgumentException("Age must be greater than 18");  
}
```

- **throws:** Used in a method signature to **declare** that a method can throw an exception. It tells the compiler that the method might throw a specific exception, which must be handled by the caller.

Example:

```
public void readFile() throws IOException {  
    // code that may throw IOException  
}
```

12) In what situations we write only one catch block and multiple catch blocks?

- **One catch block:** When we want to handle all exceptions in the same way, we can use a single catch block with a generic Exception type or a more general exception class.

```
try {  
    // Code that may throw different exceptions  
} catch (Exception e) {  
    // Handle all exceptions here  
}
```
 - **Multiple catch blocks:** When we want to handle different exceptions in different ways, we write multiple catch blocks.

```
try {  
    // Code that may throw different exceptions  
} catch (IOException e) {  
    // Handle IOException  
} catch (SQLException e) {  
    // Handle SQLException  
}
```
-

13) Is it possible to handle multiple Exceptions in a single catch block?

Yes, starting from **Java 7**, you can handle multiple exceptions in a single catch block using the **multi-catch** feature:

```
try {
```

```
// Code that may throw exceptions
} catch (IOException | SQLException e) {
    // Handle both IOException and SQLException
    e.printStackTrace();
}
```

14) What exceptions are called as Unchecked Exception?

Unchecked exceptions are subclasses of RuntimeException. They occur during runtime and are not checked at compile time. Some common unchecked exceptions are:

- NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException
 - ClassCastException
-

15) In what situation will the finally block not get executed?

The finally block is generally executed regardless of whether an exception occurs or not. However, there are a few scenarios where the finally block **won't execute**:

1. **If the JVM crashes** during the execution.
 2. **If the System.exit()** method is called, which terminates the program.
 3. **If the thread executing the finally block is interrupted** or crashes before the block is reached.
-

16) How do you create Custom Exception as checked or unchecked Exception?

- **Unchecked Exception:** Create a class that extends RuntimeException.
- `class MyUncheckedException extends RuntimeException {`

- `public MyUncheckedException(String message) {`
- `super(message);`
- `}`
- `}`
- **Checked Exception:** Create a class that extends Exception and declare it in the method signature.
- `class MyCheckedException extends Exception {`
- `public MyCheckedException(String message) {`
- `super(message);`
- `}`
- `}`

17) Explain steps to create custom/user-defined exception?

1. **Extend an existing exception class** (Exception for checked exceptions, RuntimeException for unchecked exceptions).
2. **Create a constructor** that accepts a message or other parameters.
3. **Throw the custom exception** where necessary.

Example:

```
class MyCustomException extends Exception {
    public MyCustomException(String message) {
        super(message);
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        try {
```

```
        throw new MyCustomException("Something went wrong");
    } catch (MyCustomException e) {
        System.out.println(e.getMessage());
    }
}
}
```

18) What are the important methods used to debug an exception?

Here are some common methods to help debug exceptions:

- **getMessage():** Returns the detail message of the exception.
 - **printStackTrace():** Prints the stack trace, showing the sequence of method calls that led to the exception.
 - **getCause():** Returns the cause of the exception, if any.
 - **toString():** Returns a string representation of the exception.
-

19) Differentiate between final, finally, and finalize?

- **final:** A keyword used to define constants, prevent method overriding, or prevent inheritance.
 - final class: The class cannot be subclassed.
 - final method: The method cannot be overridden.
 - final variable: The value cannot be changed.
- **finally:** A block that always executes after a try-catch block, regardless of whether an exception occurred or not.
- **finalize:** A method in Object class that is called by the garbage collector before an object is deleted.

Strings concepts

1) What is a String?

A **String** in Java is an object that represents a sequence of characters. It's a **class** in the `java.lang` package, and unlike primitive data types, it's immutable (cannot be modified once created). Strings are widely used to represent text-based data, such as names, addresses, or any other textual content.

2) What are the different ways of String object creation?

There are mainly **two ways** to create a String object:

1. **Using String Literal** (recommended for memory optimization through String Pool):
2. `String str1 = "Hello";`
3. **Using the new keyword** (creates a new object in the heap memory):
4. `String str2 = new String("Hello");`

When you use a string literal, the JVM checks if the string already exists in the **string pool**. If it does, the same reference is returned; otherwise, a new object is created and added to the pool.

3) Explain how String is immutable?

A **String** is **immutable** in Java, meaning that once it is created, its value cannot be changed. For example, if you modify a string, a new object is created instead of changing the original object. This helps in making strings thread-safe and improving performance due to string interning (reuse of identical strings).

For example:

```
String s = "hello";
```

```
s = s.concat(" world"); // A new String object is created, and "s" now refers to it.
```

In this case, even though you are concatenating, the original string "hello" remains unchanged in memory. The result of `s.concat()` is stored in a new object, and `s` now refers to that.

4) Differentiate between String, StringBuffer and StringBuilder?

Feature	String	StringBuffer	StringBuilder
Immutability	Immutable	Mutable	Mutable
Thread Safety	Not thread-safe	Thread-safe	Not thread-safe
Performance	Slower for modification due to immutability	Faster for modifications, due to mutable nature	Faster for modifications , like StringBuffer, but not thread-safe
Use Case	When the string value is not modified after creation	When the string is modified often in a multi- threaded environ- ment	When the string is modified often in a single- threaded environment

5) How do you create your own immutable object?

To create an **immutable object** in Java, follow these steps:

1. Declare the class as `final` to prevent subclassing.
2. Declare all fields as `private` and `final`.

3. Provide only getters (no setters) for the fields.
4. Ensure that fields holding mutable objects are properly cloned or wrapped to prevent modification.
5. Initialize fields via constructor only.

Example:

```
public final class Person {  
    private final String name;  
    private final int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

This class is immutable because the fields cannot be changed once set, and the class cannot be subclassed.

6) Differentiate between == and equals()?

- **==**: This is a reference comparison operator. It checks whether two references point to the **same object** in memory.
 - `String s1 = "Hello";`
 - `String s2 = "Hello";`
 - `System.out.println(s1 == s2); // true, because both reference the same object in the String pool`
 - **equals()**: This method compares the **content** of two objects. In the case of a String, it checks if the sequence of characters in both strings is the same.
 - `String s1 = "Hello";`
 - `String s2 = new String("Hello");`
 - `System.out.println(s1.equals(s2)); // true, because their contents are the same`
-

7) When is an object eligible for Garbage Collection?

An object becomes eligible for garbage collection when it is no longer reachable from any active part of the program. This typically happens when:

1. There are no references to the object.
 2. The references are set to null.
 3. The object is out of scope and cannot be accessed by any thread.
-

8) Can we explicitly invoke Garbage Collection?

Yes, we can request the JVM to run the garbage collector using:

`System.gc();`

However, calling `System.gc()` **does not guarantee** that the garbage collection will actually take place. It's just a request to the JVM, and the JVM decides when to perform it based on its internal algorithm and memory requirements.

9) Name some final inbuilt classes of Java?

In Java, some classes are declared as final to prevent them from being subclassed. Examples include:

- String
- Wrapper classes: Integer, Double, Character, etc.
- Math
- System
- Runtime

These classes are made final to ensure their integrity and prevent modification through inheritance.

10) How do you convert a String to primitive data type?

You can convert a String to a primitive data type using **parse methods**. For example:

- String to int: Integer.parseInt("123")
- String to double: Double.parseDouble("3.14")
- String to boolean: Boolean.parseBoolean("true")

Example:

```
String str = "123";
```

```
int num = Integer.parseInt(str); // Converts String to primitive int
```

11) How do you convert primitive data into String?

You can convert a primitive data type to a String using:

1. **Using String.valueOf() method:**
2. `int num = 123;`
3. `String str = String.valueOf(num);`

4. **Using Integer.toString()** for integer:

5. `int num = 123;`

6. `String str = Integer.toString(num);`

7. **Using + operator:**

8. `int num = 123;`

`String str = num + ""; // Implicitly converts to String`

Collections concepts

1) Why do we need Collection Frameworks?

The **Collection Framework** in Java provides a set of classes and interfaces for storing and manipulating data efficiently. It helps in:

- **Storing data:** Provides various data structures like lists, sets, and maps.
- **Manipulating data:** Makes operations like adding, removing, and searching elements easier.
- **Efficiency:** Optimized algorithms for handling different data types.
- **Flexibility:** Makes it easier to work with both homogeneous and heterogeneous data.
- **Interoperability:** Collection classes are part of the Java standard library, making them easy to use and integrate with various applications.

2) Write Collection Hierarchy?

Here's a simple breakdown of the Collection hierarchy in Java:

- **Collection** (root interface)
 - **Set** (no duplicates allowed)
 - **HashSet**
 - **LinkedHashSet**
 - **TreeSet**

- **List** (ordered collection)
 - **ArrayList**
 - **LinkedList**
 - **Vector**
- **Queue** (FIFO)
 - **PriorityQueue**
 - **LinkedList**
- **Deque** (double-ended queue)
 - **ArrayDeque**
 - **LinkedList**
- **Map** (associates key-value pairs)
 - **HashMap**
 - **LinkedHashMap**
 - **TreeMap**
 - **Hashtable**

3) What is the super interface of Collection Hierarchy?

The **super interface** of the collection hierarchy is **Collection**. It is the root interface in the hierarchy, from which other interfaces like List, Set, and Queue inherit.

4) Differentiate between Collection and Collections?

- **Collection**: An interface that represents a group of objects, such as List, Set, and Queue. It is part of the Java Collection Framework.
- **Collections**: A utility class in java.util that provides static methods to operate on or return collections. It contains methods like sort(), reverse(), shuffle(), etc.

5) Differentiate between List and Set?

- **List:**
 - **Ordered** collection of elements.
 - Allows **duplicates**.
 - Elements are stored at specific **indexes**.
 - Example: ArrayList, LinkedList.
- **Set:**
 - **Unordered** collection of elements.
 - Does **not allow duplicates**.
 - Does not maintain a specific order of elements.
 - Example: HashSet, LinkedHashSet.

6) Uses of Generics?

- **Type Safety:** Generics enable **type checking at compile-time**, preventing errors like ClassCastException.
- **Code Reusability:** Generic classes or methods can work with any object type, making the code more reusable.
- **Readability:** Reduces the need for casting, making code cleaner and easier to read.

Example:

```
List<String> list = new ArrayList<>();  
list.add("Hello"); // No need for type casting
```

7) How do you make Collection homogenous?

A **homogeneous** collection means it holds only one type of object. To ensure this:

- **Use Generics:** For example, `List<String>` list ensures only String objects are allowed in the list.

```
List<String> list = new ArrayList<>();
```

```
list.add("Hello");
```

```
list.add(123); // Compile-time error due to type safety
```

8) Differentiate between ArrayList and LinkedList?

Feature	ArrayList	LinkedList
Implementation	Uses a dynamic array.	Uses a doubly linked list.
Access Time	Faster for random access (O(1))	Slower for random access (O(n))
Insertion/Deletion	Slower (O(n)) when inserting/deleting in the middle	Faster (O(1)) for insertion/deletion
Memory Overhead	Lower memory overhead	Higher memory overhead due to node pointers

9) How do you Convert LinkedList to ArrayList/How do you convert one Collection to another Collection?

To convert a LinkedList to ArrayList:

```
LinkedList<String> linkedList = new LinkedList<>();
```

```
linkedList.add("A");
```

```
linkedList.add("B");
```

```
ArrayList<String> arrayList = new ArrayList<>(linkedList);
```

Similarly, you can convert any Collection to another by passing it to the constructor of the target collection.

10) Differentiate Between ArrayList and Vector?

Feature	ArrayList	Vector
Growth Factor	Grows by 50% of its size	Grows by doubling its size
Thread Safety	Not thread-safe	Thread-safe (synchronized)
Performance	Faster in single-threaded apps	Slower due to synchronization
Legacy	Part of the modern Collection Framework	Legacy class (older)

11) Explain the data Structure used in ArrayList?

ArrayList uses a **dynamic array** for storage. It maintains an internal array, which automatically grows as elements are added. The array is resized when the capacity is exceeded, usually by 50% of its current size.

12) Differentiate between add() and set()?

- **add()**: Adds an element to the collection. If the collection supports it, it adds to the end or at a specific index.

Example:

```
List<String> list = new ArrayList<>();
```

```
list.add("Hello"); // Adds "Hello" to the list
```

- **set()**: Replaces an element at a specific index in the collection with the new element.

Example:

```
list.set(0, "World"); // Replaces the element at index 0 with "World"
```

13) Differentiate between iterator and for-each loop?

- **Iterator:** An explicit object that you use to traverse a collection. It can be used with any collection (List, Set, etc.).

Example:

```
Iterator<String> iterator = list.iterator();  
while(iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

- **for-each loop:** A simplified loop for iterating through collections. It's easier to write but not as flexible as an Iterator.

Example:

```
for(String item : list) {  
    System.out.println(item);  
}
```

14) Differentiate between for and for-each loop?

- **for loop:** A general-purpose loop that can iterate over a range of values and allows modifying the loop index.

Example:

```
for(int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

- **for-each loop:** A special loop for iterating over collections and arrays without needing to manage an index.

Example:

```
for(String item : list) {  
    System.out.println(item);  
}
```


}

15) Differentiate between HashSet and LinkedHashSet?

- **HashSet:**
 - **No order:** Does not guarantee the order of elements.
 - **Faster:** Generally faster than LinkedHashSet due to lack of ordering overhead.
- **LinkedHashSet:**
 - **Maintains insertion order:** Elements are stored in the order they were inserted.
 - **Slightly slower:** Due to maintaining insertion order.

16) DataStructure of TreeSet?

TreeSet uses a **Red-Black Tree** (a self-balancing binary search tree) for storage. It maintains **sorted order** of elements and provides **$O(\log n)$** time complexity for basic operations like add, remove, and search.

17) How do you make a TreeSet to Store duplicate data?

TreeSet does not allow duplicates. To store duplicate elements, you can use a **List** or **HashSet**, but if you want to force TreeSet to store duplicates, you could use a **Comparator** that allows equality.

18) How do you make TreeSet heterogenous?

To store heterogeneous objects in a TreeSet, you'd need to use a custom comparator because TreeSet normally requires elements to be **comparable**. The comparator will handle comparing elements of different types.

19) Differentiate between Comparable and Comparator?

- **Comparable:**
 - Used to define the natural order of objects.
 - Implements compareTo() method.
 - Typically used for sorting a single class.
 - **Comparator:**
 - Used to define a custom order.
 - Implements compare() method.
 - Can be used for comparing objects of different classes.
-

20) What is Map?

A Map is an object that maps **keys** to **values**. It is not a part of the Collection hierarchy, but it's still part of the Java Collections Framework. It does not allow duplicate keys, but each key can have only one associated value.

21) Name the methods present in Entry interface?

The **Map.Entry** interface provides methods to interact with key-value pairs:

- getKey(): Returns the key.
 - getValue(): Returns the value.
 - setValue(V value): Sets the value.
-

22) Differentiate between HashMap and LinkedHashMap?

- **HashMap:** Does not maintain any order of its keys.
 - **LinkedHashMap:** Maintains insertion order of keys.
-

23) Differentiate between HashMap and Hashtable?

- **HashMap:** Not synchronized, allows one null key, and multiple null values.
- **HashTable:** Synchronized, does not allow null keys or values.